Case 02 · Design document: Private Exposure Registry

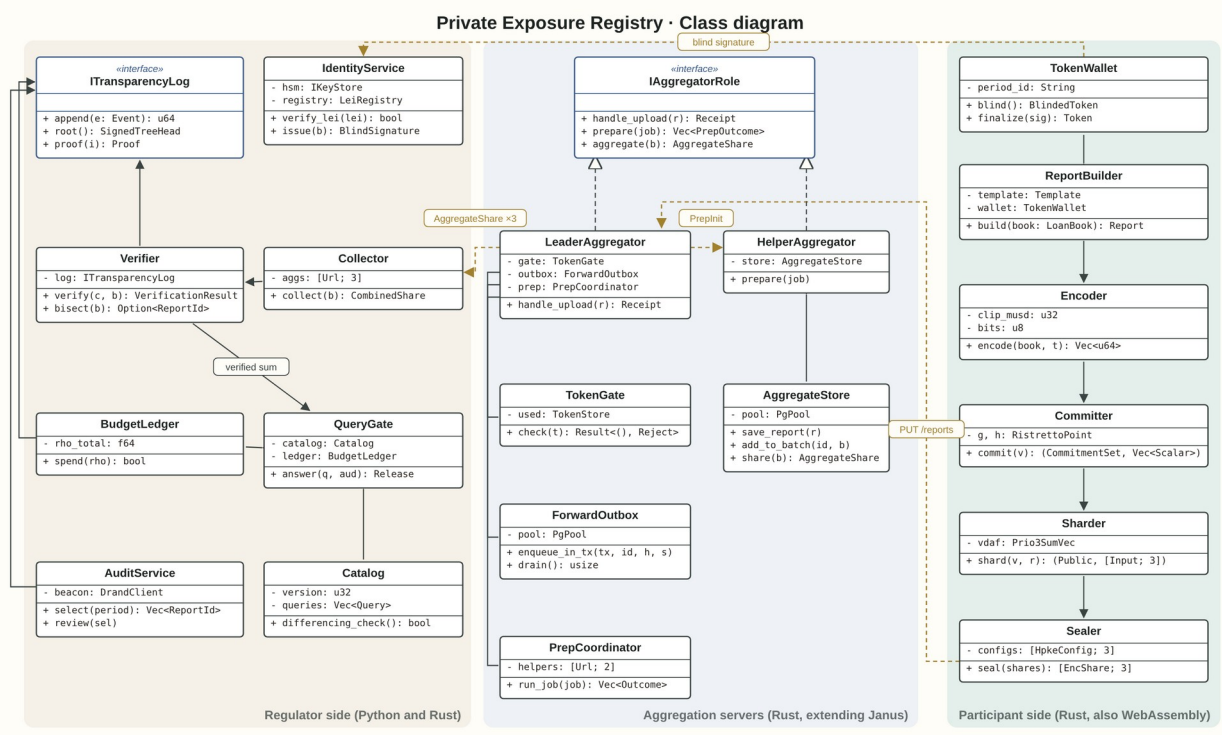
The stage after architecture. The architecture decided which components exist and what passes between them. Design goes one level down: which classes live inside each component, which states each object passes through, who talks to whom and in what order, which threads run in parallel, and what the tables look like. The structure matches the Case 01 design document and the Trivia server document.

Design principles

Four principles settle almost every decision in this document. When in doubt about how to build something, we come back to them.

- **No object holds both identity and content:** the class that knows who the participant is (IdentityService) never sees a report, and the class that receives reports (LeaderAggregator) does not know who sent them. This separation is not a procedure; it is enforced by structure: there is no field, no parameter and no table that links the two.
- **The log comes first:** every significant state change is first written as an event to the transparency log, and only then is the state updated. If state and log disagree, the log is right (Event Sourcing, as decided in the architecture).
- **An interface at every trust boundary:** wherever we cross between parties that do not trust each other, there is an abstract interface (IAggregatorRole, ITransparencyLog). This lets us swap an implementation, test with a fake, and add a third party without touching the rest.
- **Fail closed:** an unverified sum is not available to any query. There is no "almost verified" state. Every branch of every state machine that does not end in verification ends in quarantine or rejection.

Class diagram



The diagram is divided into three areas, like the three kinds of party in the architecture: the participant side (a client library the reporting institution runs itself), the aggregation servers (three instances, one at each authority), and the regulator (the layer that identifies, verifies, answers queries and audits).

The labelled dashed arrows are network messages, not function calls: they cross a trust boundary, so each of them is encrypted and signed.

The role of each class

Class	Area	Role	What it does not do
TokenWallet	Participant	Holds blind tokens issued in advance, one per reporting period	Does not know which report a token will be attached to until the moment of sending
ReportBuilder	Participant	Collects the 33 cells from the institution's internal systems and checks ranges	Does not encrypt or send
Encoder	Participant	Converts the cells into finite-field integers per the template, for Prio3SumVec and Prio3Histogram	Knows nothing about the servers
Committer	Participant	Creates a Pedersen commitment on Ristretto255 for each cell, and keeps the randomness	Never shares the randomness with anyone in the clear
Sharder	Participant	Splits into three random shares that sum to the data, and attaches a range proof	Does not send
Sealer	Participant	Encrypts each share with its server's HPKE key (RFC 9180)	Cannot see the content after encryption
IAggregatorRole	Aggregation	Common interface for leader and helpers: receive a share, prepare, aggregate	
LeaderAggregator	Aggregation	Receives the report, forwards shares to the helpers, runs aggregation jobs	Holds no decryption key of the helpers
HelperAggregator	Aggregation	Decrypts its own share, checks the proof, aggregates	Does not receive reports directly from participants
TokenGate	Aggregation	Verifies a blind token and marks it as spent	Does not know who received the token
ForwardOutbox	Aggregation	Stores forwards to helpers in the same transaction as receipt, and resends until success	Does not decide whether a report is valid
PrepCoordinator	Aggregation	Runs the VDAF preparation stage with	

Class	Area	Role	What it does not do
		the helpers and decides each report: accepted or rejected	
AggregateStore	Aggregation	Holds this server's partial sum for each batch	Cannot decrypt another server's sum
IdentityService	Regulator	Verifies LEI against GLEIF and issues blind tokens per Privacy Pass	Never sees reports
Verifier	Regulator	Compares the received sum with the sum of commitments; if they differ, bisects	
Collector	Regulator	Collects the three partial sums and adds them	Never sees an individual report
BudgetLedger	Regulator	Manages the privacy budget (zCDP) for each batch	
QueryGate	Regulator	Takes a query from the catalog, checks the K_MIN threshold and differencing, adds Discrete Gaussian noise	Does not answer a query that is not in the catalog
AuditService	Regulator	Draws participants for audit by drand and asks them to open a commitment	Does not choose at its own discretion
Catalog	Regulator	A closed list of permitted queries, approved with three keys	
ITransparencyLog	Shared	Interface to the Trillian log: append event, inclusion proof, consistency proof	

Message protocol

As with the Trivia server, every message is defined in advance: who sends it, what it contains, and the possible responses. The protocol rests on DAP (draft-ietf-ppm-dap-19), so most messages are existing DAP messages, with two extensions: the blind token and the commitments.

Participant to leader

```
PUT /tasks/{task_id}/reports
Content-Type: application/dap-report
Body:
  report_metadata { report_id: 16 bytes, time: uint64 }
  public_share    { commitments: 33 × 32 bytes }      extension
  token           { blind_token: Privacy Pass }       extension
  leader_share    HPKE ciphertext
  helper_shares   [ HPKE ciphertext × 2 ]
```

Response	Meaning	What the participant does
201 Created + Receipt	The report was accepted and logged. The Receipt includes an inclusion proof	Keeps the Receipt as evidence for the regulator
400 invalid_token	The token is invalid or not from this period	Requests a new token from the IdentityService
409 duplicate	The token has already been spent	Does not resend: the earlier report was accepted
410 period_closed	The period has closed	Contacts the regulator through the ordinary channel

Leader to helper

```
PUT /tasks/{task_id}/aggregation_jobs/{job_id}
Body: AggregationJobInitReq { prep_inits: [ PrepInit{ report_share,
leader_prep_share } ] }
Resp: AggregationJobResp   { prep_resps: [ PrepResp{ report_id, continue |
reject(reason) } ] }
```

With three servers, the leader runs preparation with both helpers, and a report is accepted only if all three agree the proof is valid.

Regulator to the servers

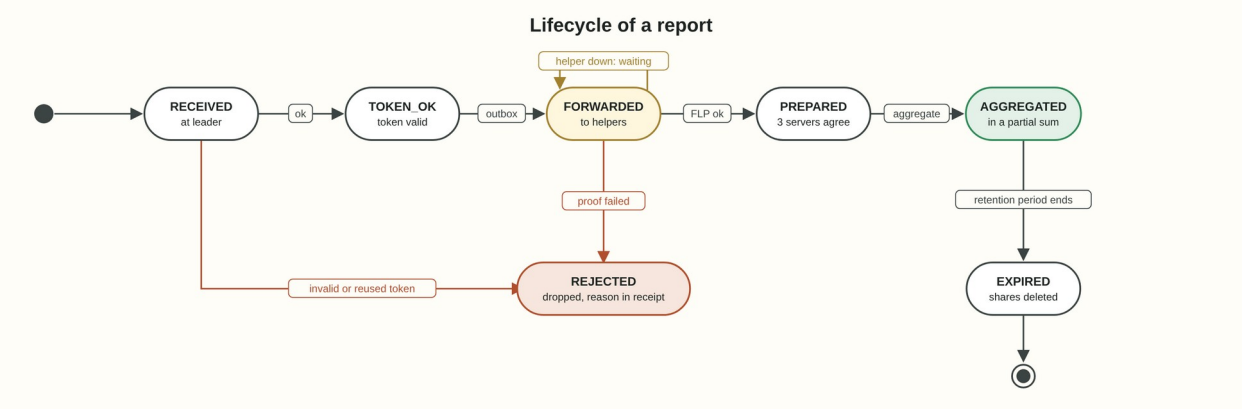
```
POST /tasks/{task_id}/collection_jobs/{collect_id}
Body: { batch_id, aggregation_param }
Resp: Collection { report_count, interval, encrypted_agg_shares: [ × 3 ] }
```

Log events

Event	Written by	When	Contents
ReportReceived	LeaderAggregator	Report accepted	report_id, hash of the commitments, time. Not who sent it
ReportRejected	PrepCoordinator	Proof failed	report_id, reason
BatchClosed	LeaderAggregator	End of period	batch_id, number of reports
BatchVerified	Verifier	Sum matches commitments	batch_id, hash of the sum
BatchVerificationFailed	Verifier	Mismatch	batch_id, reports in quarantine
CatalogApproved	Catalog	New catalog version	version, three signatures
QueryAnswered	QueryGate	A query answered	query_id, catalog version, noised result
BudgetSpent	BudgetLedger	After every answer	batch_id, ρ spent, remaining
AuditSelected	AuditService	A draw	drand round, selected participants

State machines

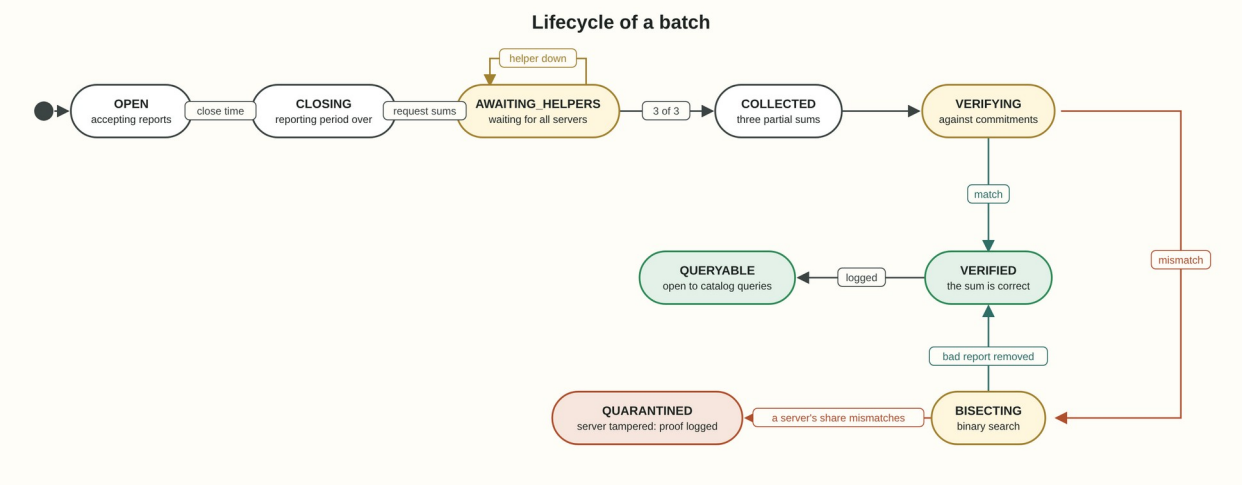
Lifecycle of a report



The report is the system's central object, and its state is kept in the reports table at the leader. A few things worth noticing:

- The first check is the token, before any decryption. So a flooding attack costs the attacker a valid token per report, and costs the server almost nothing.
- **The self-loop on FORWARDED is the Outbox:** if a helper is unavailable, the report is neither rejected nor lost. It waits, and the forward is retried until it succeeds.
- REJECTED is a final state, logged with a reason. A rejected report enters no sum.
- **EXPIRED:** once the batch has been verified and stored, the individual shares are deleted. There is no reason to keep them, and every day they exist is a risk.

Lifecycle of a batch

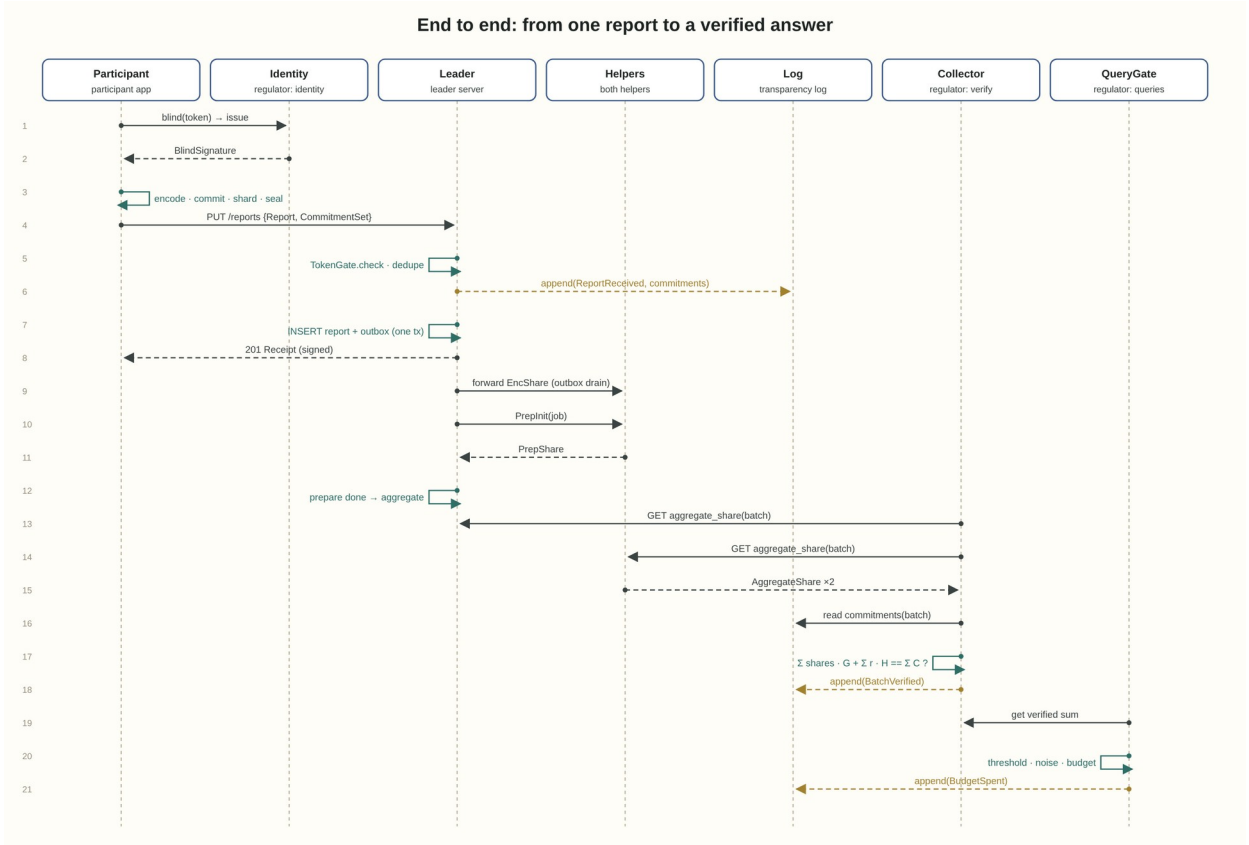


The batch is the object the regulator sees. It opens at the start of a reporting period and reaches QUERYABLE only after full verification.

- **VERIFYING:** the Verifier computes the sum of all commitments and checks that it commits to the received sum. This is the check that guarantees no server added or dropped a report.
- **BISECTING:** if the check fails, the batch is split in two and each half is checked, until the faulty reports are found. With 500 participants, that is at most 9 rounds.

- **QUARANTINED:** if the search finds no clear source, the whole batch is quarantined, and the log event notifies all three authorities. This is the fail-closed principle.

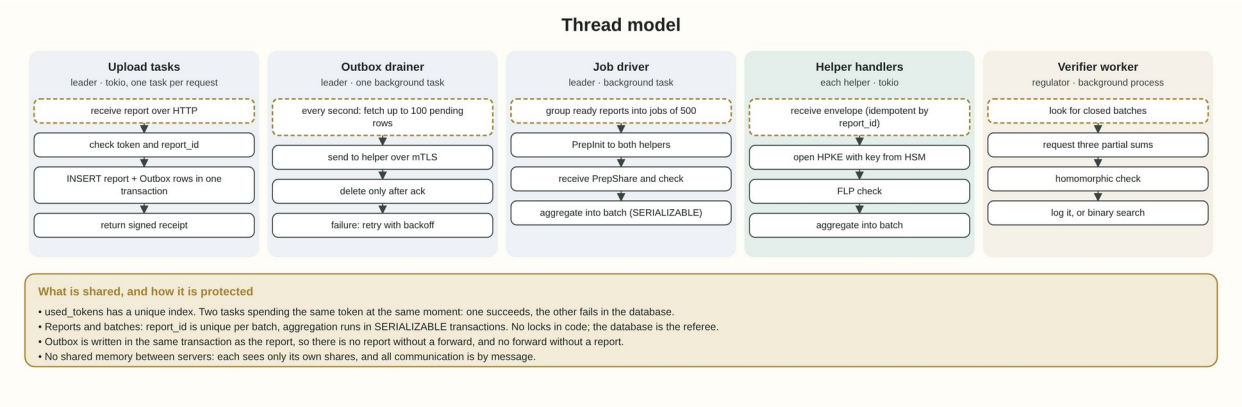
Sequence diagram



The sequence shows one reporting period end to end: from issuing the token until the budget is recorded. Three important points:

- Steps 1 to 3 happen long before reporting. The time gap between issuing a token and using it is part of the protection: even if the IdentityService and the TokenGate colluded, they could not link a token to a report by timing.
- ReportReceived is written to the log before the participant receives 201. If the log write fails, the participant gets an error and resends.
- Noise is added only at query time, not during aggregation. The verified sum stays exact, and only what goes out is noised. So we can verify first, and protect afterwards.

Thread model



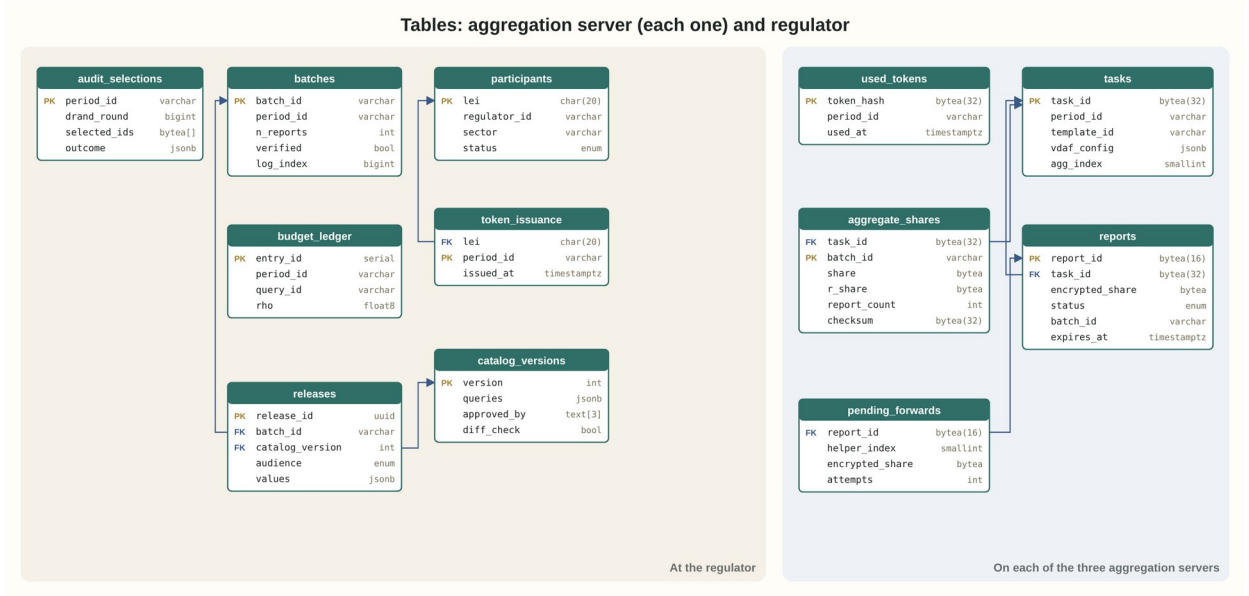
Each aggregation server is a Rust process (Janus) on the tokio runtime. That means a few operating-system threads, with thousands of asynchronous tasks running on them.

Thread or task	How many	What it does	How it synchronises
Upload tasks	One per request	Token check, storing the report, writing to the Outbox	One PostgreSQL transaction
Outbox drainer	One per server	Fetches pending forwards and sends them to helpers	SELECT ... FOR UPDATE SKIP LOCKED
Job driver	A fixed pool	Creates aggregation jobs and runs preparation	A lease on the table row
Helper handlers	One per request	Preparation and aggregation at the helper	A transaction per job
Verifier worker	One at the regulator	Verifying closed batches	A queue of events from the log

Four rules that prevent conflicts

- **used_tokens with a unique index:** two requests with the same token arrive in the same millisecond. One gets in, the other fails on the index and receives 409. No lock in code is needed.
- **SERIALIZABLE on partial-sum updates:** two jobs adding to the same batch will not overwrite each other.
- **The Outbox in the same transaction:** the report and the forward to the helpers are stored together or not at all. There is no state in which a report was accepted but will never reach the helpers.
- **No shared memory between servers:** the three servers talk only by message. This is not just a performance choice; it is a privacy requirement.

Table design



Two completely separate databases, with no foreign key between them. This is the first design principle, translated into tables: no table at the regulator contains a report_id, and no table on the servers contains an LEI.

On each aggregation server

Table	What is stored	Note
tasks	The task definition: VDAF, HPKE keys, period length	Loaded once at setup
reports	Report by report_id: encrypted share, state, commitments	Deleted on the move to EXPIRED
used_tokens	Hash of every spent token	Unique index. Not deleted until the period ends
aggregate_shares	The partial sum for each batch and the report count	What the regulator collects
pending_forwards	The Outbox: forwards not yet acknowledged	A row is deleted only after a 200 from the helper

At the regulator

Table	What is stored	Note
participants	LEI, supervising authority, status	No report data at all
token_issuance	How many tokens each participant was issued per period	Not the tokens themselves: they are blind
catalog_versions	Catalog versions, three signatures	Only a signed version is active
batches	Batch state, the verified sum	Reaches QUERYABLE only after verification
budget_ledger	ρ spent per batch	Append only, never updated
releases	Released answers, with noise	The next query checks differencing against this table
audit_selections	drand round and selected participants	Reproducible by anyone

Implementation order by versions

Each version stands on its own and is tested before moving on. The order follows the same rule as Case 01: first the core, without which the rest is pointless, then what protects it, and finally what uses it.

Version	What is built	Exit test
V0.1	Two-server core on Janus and libprio, one report and one sum	500 synthetic reports, the sum equals the direct sum
V0.2	Three servers, Outbox, HPKE encryption	A helper switched off midway: no report is lost
V0.3	Commitments, Verifier and bisection	A server deliberately drops a report: caught, and the report found
V0.4	LEI identity, blind tokens, TokenGate	Same token twice: 409. A forged token: 400
V0.5	Trillian transparency log	Every event with an inclusion proof, and consistency between two points in time
V0.6	Catalog, QueryGate, noise and budget	Two queries whose difference exposes a participant: the second is blocked
V1.0	Regulator reports, random audit, SDMX export	A full reporting period, end to end, with three authorities

What this document closes

After the design document, every class knows what it does and what it does not do, every object knows where it may move, and every table knows what it may store. The Registry's central promise, to combine without seeing, is translated here into concrete lines: no field links identity to content, and no state transition leads to an unverified sum.